MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report
of
Database Systems Lab (CSS 2212)

Hostel Allotment System

SUBMITTED
BY

NAME	REGISTRATION NO.	ROLL NO.	SECTION
Sirish Saraf	240911616	54	IT-B2
Adhyayan Rana	240911378	35	IT-B2
Aryan Devpura	240911108	12	IT-B1

School of Computer Engineering
Manipal Institute of Technology, Manipal.

April 2026

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the project titled **Hostel Allotment System** is a record of the bonafide work done by **Adhyayan Rana (240911378), Sirish Saraf (240911616), Aryan Devpura (240911108)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in Information Technology of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Name and Signature of Examiners:

1. **Dr. Jashma Suresh, Associate Professor, SCE.**

2. **Sheena KS, Assistant Professor, Department of Computer Science and Engineering , SCE.**

TABLE OF CONTENTS

SR. NO.	Chapter	Page No.
	Abstract	1
1.	Introduction	2
2.	Problem Statement & Objectives	4
3.	Methodology	6
4.	ER Diagram & Relational Tables along with sample data	8
5.	DDL commands & PL/SQL Procedures / Functions / Triggers	18
6.	Results & Snapshots	32
7.	Conclusion, Limitations & Future Work	38
8.	References	39

ABSTRACT

Hostel accommodation management in educational institutions involves handling large volumes of data related to students, room availability, allocation preferences, and administrative operations. Manual or semi-automated systems often lead to inefficiencies such as improper room allocation, conflicts in preferences, lack of transparency, and difficulty in managing student groups.

This project presents the design and implementation of a Hostel Management System that focuses on efficient and automated hostel room allocation using a relational database approach. The system enables structured storage and management of student data, room details, allocation sessions, and preferences while ensuring data integrity and consistency.

The backend is designed using a relational database system implemented through SQL and managed via Prisma ORM, ensuring normalized schema design and efficient querying. The system supports features such as student registration, group formation, preference-based room selection, and automated allotment across multiple rounds. Administrative functionalities include session management, bulk data upload using CSV files, validation mechanisms, and result generation.

Key database features include multiple normalized tables, enforcement of constraints (Primary Keys, Foreign Keys, UNIQUE constraints), and implementation of complex queries for allocation logic and reporting. The system demonstrates practical application of core DBMS concepts such as normalization, relational schema design, transaction handling, and query optimization in a real-world scenario.

1. INTRODUCTION

Educational institutions face significant challenges in managing hostel accommodations due to increasing student numbers and diverse allocation requirements. Traditional manual systems or basic software tools often fail to handle complex scenarios such as group allocations, preference-based room selection, and fair distribution of resources.

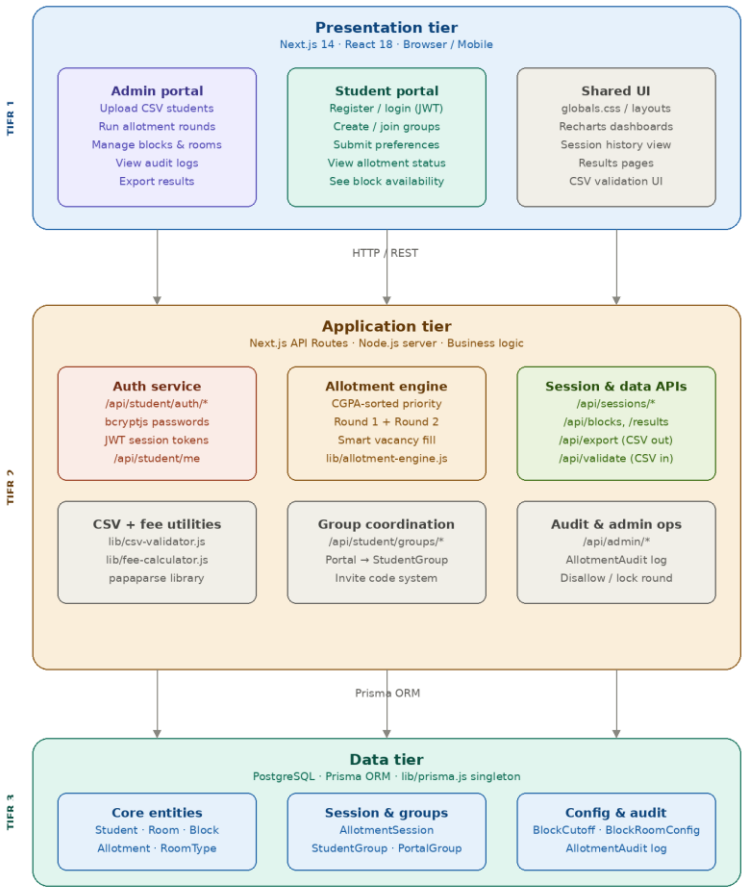
The Hostel Management System is designed to address these challenges by providing a structured and scalable database-driven solution. The system automates the process of student registration, room allocation, and administrative control while ensuring data consistency and integrity through a relational database model.

The project emphasizes the application of core Database System (DBS) principles including entity-relationship modeling, normalization, relational schema design, and implementation of SQL queries for efficient data manipulation. The system is capable of handling multiple allocation rounds, managing student preferences, and supporting group-based allotments.

The backend database is designed using SQL and integrated with Prisma ORM for structured data access and manipulation. The application logic processes allocation rules and preferences while maintaining strict adherence to database constraints. This ensures that issues such as duplicate allocations, overbooking, and inconsistent data are avoided.

1.1. System Architecture

The system follows a modern three-layered architecture consisting of a frontend layer, an application layer, and a database layer. The frontend layer is built using Next.js and is responsible for handling user interactions and providing an intuitive interface for both students and administrators. The application layer manages the core business logic of the system, including operations such as room allocation, preference processing, and data validation. The database layer is implemented using a relational database managed through SQL and Prisma ORM, which ensures structured data storage and efficient query execution. This layer serves as the backbone of the system, ensuring that all operations such as student registration, preference storage, and room allocation are performed reliably, consistently, and with maintained data integrity.



1.2. Scope of the Project

The scope of this project includes:

- Design of a normalized relational database schema
- Implementation of student, hostel, room, and allocation management
- Handling of preference-based and group-based room allocation
- Execution of complex SQL queries for allocation and reporting
- Support for administrative operations such as session control and data upload
- Demonstration of DBS concepts including constraints, joins, and transactions

1.3. Tools and Technologies

- Database: SQL (Relational Database)
- ORM: Prisma ORM
- Frontend: Next.js (React)
- Backend Logic: Server-side APIs (Next.js)
- Data Handling: CSV upload and validation
- Development Tools: Antigravity / VS Code / GitHub
- IDE and Code Support: Antigravity / VS Code
- Database Hosting: Neon Console

2. PROBLEM STATEMENT AND OBJECTIVES

2.1. Problem Statement

Managing hostel accommodation in educational institutions is a complex task involving multiple entities such as students, rooms, blocks, and administrative rules. Traditional manual systems or basic digital solutions often fail to efficiently handle large datasets, leading to issues such as duplicate allocations, inefficient utilization of rooms, lack of transparency, and difficulty in managing student preferences.

One of the major challenges is implementing a fair and efficient allocation mechanism that considers student preferences, group formations, and room availability while maintaining data consistency. Additionally, handling bulk student data, validating inputs, and supporting multiple rounds of allocation further increases system complexity.

Without a structured relational database system, the following problems may arise:

- Redundant and inconsistent data storage
- Duplicate student registrations or allocations
- Over-allocation of rooms beyond capacity
- Difficulty in managing group-based allocations
- Inefficient querying and reporting of allocation results
- Lack of validation during bulk data uploads
- Absence of a transparent and reproducible allocation process

Therefore, there is a need for a database-driven system that can efficiently manage hostel data, enforce constraints, and support complex allocation logic using structured queries.

2.2. Functional Requirements

- **Student Registration and Authentication**

  The system must allow students to register and log in securely. Each student must have a unique identity in the database.

- **Group Formation**

  Students must be able to create or join groups. The system must enforce constraints such as maximum group size and prevent invalid group configurations.

- **Hostel and Room Management**

  The system must maintain structured data for hostel blocks and rooms, including room capacity and availability.

- **Preference Submission**
  Students must be able to submit preferences for hostel blocks and rooms which must be stored in a structured and queryable format.
- **Allocation Engine**
  The system must allocate rooms based on cgpa and preferences of an individual or a group, room availability and allocation rules and constraints.
- **Bulk Data Upload**
  Administrators must be able to upload student data using CSV files. The system must validate data before inserting it into the database.
- **Data Validation and Integrity**
  The system must enforce unique constraints, referential integrity and valid input formats.
- **Reporting and Results**
  The system must generate allocation results and allow administrators to view and export data.

2.3. Objectives

- Design a normalized relational database schema to manage hostel operations efficiently. The schema has been normalized upto Third Normal Form (3NF).
- Implement tables with appropriate primary and foreign keys, NOT NULL, UNIQUE and CHECK constraints.
- Ensure data integrity, validity and consistency through constraint enforcement.
- Develop SQL queries for data insertion and updates, allocation processing and result generation.
- Support group-based and preference-based allocation logic.
- Enable bulk data handling and validation using structured formats.
- Demonstrate use of joins, subqueries, and aggregation in real-world scenarios.
- Provide a scalable system capable of handling large datasets

3. METHODOLOGY

This project follows the Database Development Life Cycle (DDLC) to ensure systematic design and implementation

3.1. Requirements Elicitation and Analysis

The system requirements were identified by analyzing common challenges faced in hostel allocation systems. Key entities such as students, rooms, hostels, groups, and preferences were identified along with their relationships. Functional requirements such as registration, group formation, preference handling, and allocation were defined. Necessary constraints and business rules were also established to ensure data integrity and consistency within the system.

3.2. Conceptual Design

An Entity-Relationship model was designed to represent the system. 13 major entities and relationships among them were designed to represent real-world components involved in a hostel allocation system.

3.3. Logic Design & Normalization

The ER model was converted into a relational schema and normalized upto the Third Form (3NF) to reduce redundancy and improve data consistency.

- **First Normal Form (1NF):**
  All attributes are atomic and no repeating groups exist.
- **Second Normal Form (2NF):**
  All non-key attributes are fully dependent on the primary key.
- **Third Normal Form (3NF):**
  No transitive dependencies exist and all attributes depend only on the primary key.

3.4. Physical Design

The database was implemented using a relational SQL database and managed through Prisma ORM. All the tables are created with appropriate primary and foreign keys, UNIQUE, NOT NULL and CHECK constraints. Relations were also well-defined between the tables wherever applicable.

3.5. Allocation Algorithm

The allotment engine is a greedy, priority-based algorithm that assigns student groups to rooms based on preferences, capacity, and eligibility constraints. Groups are first sorted by average CGPA and academic year, and allocation is performed in two rounds to maximize accommodation efficiency. In Round 1, the system attempts to allocate rooms based on student preferences, with an optional smart vacancy mechanism that fills available rooms using optimized criteria. Groups that cannot be accommodated are waitlisted and reprocessed in Round 2, where all possible allocation strategies are applied. The algorithm operates entirely in memory and commits results to the database in a single transaction, ensuring consistency and efficient execution.

3.6. Implementation

The system was implemented using a relational SQL database for data storage and management, with Prisma ORM serving as the interface for database interaction. Prisma ORM was used instead of raw SQL to simplify database interaction and improve code maintainability while still leveraging the power of a relational database. It provides an abstraction layer that allows developers to define the database schema and perform operations using structured, type-safe queries, reducing the chances of errors compared to writing raw SQL manually. Prisma also handles complex operations such as joins, filtering, and transactions internally, enabling faster development and cleaner code organization. Additionally, it ensures consistency between the application and the database schema through migrations, making it easier to manage changes and maintain data integrity throughout the project. The application logic is handled through API routes, which process user requests and coordinate interactions between the frontend and the database. Key operations implemented in the system include CRUD (Create, Read, Update, Delete) operations for all entities, structured storage and retrieval of student preferences, allocation processing based on defined logic and constraints, and bulk data insertion with appropriate validation mechanisms to ensure data integrity.

3.7. Testing & Validation

The system was tested to ensure correct enforcement of constraints to prevent invalid data insertion and updates, proper functioning of the allocation logic and accurate results. Validation checks were performed to prevent duplicate entries, incorrect input formats and invalid foreign key references.

4. ER Diagram & Relational Tables along with sample data

4.1. Entity-Relationship Overview

The Hostel Management System is designed using a relational model consisting of multiple interconnected entities that represent real-world components of hostel allocation.

4.2. Relational Schema

ALLOTMENT_SESSIONS [Strong Entity]

Core session entity — central hub for all allotment activity

ALLOTMENT_SESSIONS(id PK, name, status, createdAt, algoMode, portalStatus, blockCutoffsEnabled, maxCgpaDiffEnabled, maxCgpaDiff)

Attribute	Type	Key	Null	Notes
id	integer	PK		
name	varchar			
status	varchar			default: 'DRAFT'
createdAt	timestamp			
algoMode	varchar			default: 'preference_only'
portalStatus	varchar			default: 'CLOSED'
blockCutoffsEnabled	boolean			default: false
maxCgpaDiffEnabled	boolean			default: false
maxCgpaDiff	float		Y	nullable

STUDENTS [Strong Entity]

Student records scoped to a session

STUDENTS(id PK, regNo UK, name, gender, year, department, cgpa, sessionId FK, pref1BlockNum, pref1RoomTypeCode, pref2BlockNum, pref2RoomTypeCode, password, source)

Attribute	Type	Key	Null	Notes
id	integer	PK		
regNo	varchar	UK		unique per session
name	varchar			
gender	varchar			
year	integer			
department	varchar			
cgpa	float			
sessionId	integer	FK		→ allotment_sessions.id
pref1BlockNum	integer		Y	nullable
pref1RoomTypeCode	varchar		Y	nullable
pref2BlockNum	integer		Y	nullable
pref2RoomTypeCode	varchar		Y	nullable
password	varchar			
source	varchar			default: 'CSV'

BLOCKS [Strong Entity]

Physical hostel blocks

BLOCKS(id PK, number UK, gender, floors, roomsPerFloor)

Attribute	Type	Key	Null	Notes
id	integer	PK		
number	integer	UK		unique block number
gender	varchar			
floors	integer			default: 8
roomsPerFloor	integer			default: 30

ROOM_TYPES [Strong Entity]

Room type definitions (single, double, etc.)

ROOM_TYPES(id PK, code UK, name, capacity)

Attribute	Type	Key	Null	Notes
id	integer	PK		
code	varchar	UK		
name	varchar			
capacity	integer			

ROOMS [Strong Entity]

Individual rooms inside blocks

ROOMS(id PK, blockId FK, roomTypeId FK, floor, roomNumber UK, occupiedBeds)

Attribute	Type	Key	Null	Notes
id	integer	PK		
blockId	integer	FK		→ blocks.id
roomTypeId	integer	FK		→ room_types.id
floor	integer			
roomNumber	varchar	UK		unique per block
occupiedBeds	integer			default: 0

STUDENT_GROUPS [Strong Entity]

Algorithm-facing groups with averaged preferences

STUDENT_GROUPS(id PK, avgCgpa, size, pref1BlockId FK, pref1RoomTypeId FK, pref2BlockId FK, pref2RoomTypeId FK, sessionId FK)

Attribute	Type	Key	Null	Notes
id	integer	**PK**		
avgCgpa	float			
size	integer			
pref1BlockId	integer	**FK**		→ blocks.id
pref1RoomTypeId	integer	**FK**		→ room_types.id
pref2BlockId	integer	**FK**	Y	→ blocks.id, nullable
pref2RoomTypeId	integer	**FK**	Y	→ room_types.id, nullable
sessionId	integer	**FK**		→ allotment_sessions.id

PORTAL_GROUPS [Strong Entity]

Student-facing groups created via invite code

PORTAL_GROUPS(id PK, sessionId FK, inviteCode UK, leaderId FK, pref1BlockId FK, pref1RoomTypeId FK, pref2BlockId FK, pref2RoomTypeId FK, isSubmitted, createdAt)

Attribute	Type	Key	Null	Notes
id	integer	**PK**		
sessionId	integer	**FK**		→ allotment_sessions.id
inviteCode	varchar	**UK**		
leaderId	integer	**FK**		→ students.id
pref1BlockId	integer	**FK**		→ blocks.id
pref1RoomTypeId	integer	**FK**		→ room_types.id
pref2BlockId	integer	**FK**	Y	→ blocks.id, nullable
pref2RoomTypeId	integer	**FK**	Y	→ room_types.id, nullable
isSubmitted	boolean			default: false
createdAt	timestamp			

ALLOTMENTS [Relationship Table]

Room allotment records per student per round

ALLOTMENTS(id PK, studentId FK, roomId FK, groupId FK, round, status, sessionId FK)

Attribute	Type	Key	Null	Notes
id	integer	PK		
studentId	integer	FK		→ students.id
roomId	integer	FK		→ rooms.id
groupId	integer	FK		→ student_groups.id
round	integer			
status	varchar			
sessionId	integer	FK		→ allotment_sessions.id

GROUP_MEMBERS [Relationship Table]

Resolves M:N between student_groups and students

GROUP_MEMBERS(id PK, groupId FK, studentId FK)

Attribute	Type	Key	Null	Notes
id	integer	PK		
groupId	integer	FK		→ student_groups.id
studentId	integer	FK		→ students.id

PORTAL_GROUP_MEMBERS [Relationship Table]

Resolves M:N between portal_groups and students

PORTAL_GROUP_MEMBERS(id PK, portalGroupId FK, studentId FK)

Attribute	Type	Key	Null	Notes
id	integer	PK		
portalGroupId	integer	FK		→ portal_groups.id
studentId	integer	FK		→ students.id

BLOCK_CUTOFFS [Weak Entity]

Per-session per-block minimum CGPA cutoffs

BLOCK_CUTOFFS(id PK, sessionId FK, blockId FK, minCgpa)

Attribute	Type	Key	Null	Notes
id	integer	PK		
sessionId	integer	FK		→ allotment_sessions.id
blockId	integer	FK		→ blocks.id
minCgpa	float			

BLOCK_ROOM_CONFIGS [Weak Entity]

Rooms-per-floor config for each block+room type pair

BLOCK_ROOM_CONFIGS(id PK, blockId FK, roomTypeId FK, roomsPerFloor)

Attribute	Type	Key	Null	Notes
id	integer	PK		
blockId	integer	FK		→ blocks.id
roomTypeId	integer	FK		→ room_types.id
roomsPerFloor	integer			

ALLOTMENT_AUDITS [Weak Entity]

Audit trail of student allotment status changes

ALLOTMENT_AUDITS(id PK, studentId FK, oldStatus, newStatus, changedAt)

Attribute	Type	Key	Null	Notes
id	integer	PK		
studentId	integer	FK		→ students.id
oldStatus	varchar		Y	nullable
newStatus	varchar			
changedAt	timestamp			

Relationship Cardinalities

All binary relationships between tables, with cardinality ratio and participation type.

Relationship	Cardinality	Participation	Notes
allotment_sessions → students	1 : N	Total	Every student row must belong to a session (sessionId NOT NULL)
allotment_sessions → allotments	1 : N	Total	Every allotment belongs to exactly one session
allotment_sessions → student_groups	1 : N	Total	Groups are scoped to a session
allotment_sessions → portal_groups	1 : N	Total	Portal groups are scoped to a session
allotment_sessions → block_cutoffs	1 : N	Total	Cutoffs exist only within a session context
blocks → rooms	1 : N	Total	Every room must belong to a block (blockId NOT NULL)
blocks → block_room_configs	1 : N	Total	Configs always reference a valid block
blocks → block_cutoffs	1 : N	Partial	A block may have no cutoffs in a given session
room_types → rooms	1 : N	Total	Every room must have a type (roomTypeId NOT NULL)
room_types → block_room_configs	1 : N	Partial	A room type may not appear in every block config
students ↔ student_groups	M : N	Partial	Via group_members; a student may not be in any group
students ↔ portal_groups	M : N	Partial	Via portal_group_members; membership is optional
students → allotments	1 : N	Partial	A student may not yet have an allotment
students → allotment_audits	1 : N	Partial	A student may have no audit entries yet
students → portal_groups (leader)	1 : N	Partial	A student may not lead any portal group
rooms → allotments	1 : N	Partial	A room may be unoccupied (no allotment row)
student_groups → allotments	1 : N	Partial	A group may not have been allotted yet
portal_groups → portal_group_members	1 : N	Total	Every member row references a valid portal group

Total vs Partial Participation

A relationship has total participation when every entity instance must participate; partial when participation is optional.

Total Participation

The following entities have mandatory participation — the foreign key is NOT NULL and every instance must be associated.

Entity / Attribute	Reason
students → allotment_sessions	students.sessionId is NOT NULL — every student must belong to a session
rooms → blocks	rooms.blockId is NOT NULL — every room must be in a block
rooms → room_types	rooms.roomTypeId is NOT NULL — every room must have a type
allotments → allotment_sessions	allotments.sessionId, studentId, roomId are all non-nullable
allotments → students	allotments.studentId is NOT NULL
allotments → rooms	allotments.roomId is NOT NULL
group_members → student_groups	groupId is NOT NULL — every member row ties to a group
group_members → students	studentId is NOT NULL — every member row ties to a student
block_cutoffs → allotment_sessions	sessionId is NOT NULL — cutoffs always scoped to a session
block_cutoffs → blocks	blockId is NOT NULL — cutoffs always tied to a block
portal_group_members → portal_groups	portalGroupId is NOT NULL
portal_group_members → students	studentId is NOT NULL
allotment_audits → students	studentId is NOT NULL — every audit row tracks a student

Partial Participation

The following entities or attributes have optional participation — the relationship may or may not exist for a given instance.

Entity / Attribute	Reason
students in allotments	A student may not yet have an allotment row (allotment is optional)
rooms in allotments	A room can exist without being allotted (occupiedBeds may be 0)
student_groups in allotments	A group may be formed but not yet allotted
students in group_members	A student may not be part of any student group
students in portal_group_members	A student may not have joined any portal group
students in portal_groups (leader)	A student need not be the leader of any portal group
blocks in block_cutoffs	A block may have no CGPA cutoffs in a particular session
room_types in block_room_configs	A room type may not be configured for every block
allotment_sessions.maxCgpaDiff	Nullable — only meaningful when maxCgpaDiffEnabled = true
student_groups.pref2BlockId	Nullable — second preference block is optional
student_groups.pref2RoomTypeId	Nullable — second preference room type is optional
portal_groups.pref2BlockId	Nullable — second preference block is optional

4.3. Sample Data

- STUDENTS

id	regNo	name	gender	year	department	cgpa	sessionId	pref1BlockNum	pref1RoomTypeCode	pref2BlockNum	pref2RoomTypeCode	password	source
20006	240901001	Pranav Verma	MALE	2	CSE	8.1	6	23	QAC	23	QAC		CSV
20007	240912002	Arjun Chatterjee	MALE	2	AI	7.33	6	23	QAC	23	QAC		CSV
20008	240907003	Aman Sharma	MALE	2	CE	8.28	6	23	QAC	23	QAC		CSV
20009	240904004	Raghav Pillai	MALE	2	EE	9.33	6	23	QAC	23	QAC		CSV
20010	240904005	Tarun Menon	MALE	2	EE	7.22	6	23	QAC	23	QAC		CSV
20011	240910006	Pranav Sharma	MALE	2	CHE	7.22	6	23	QAC	23	QAC		CSV
20012	230903007	Manish Agarwal	MALE	3	ME	9.4	6	23	QAC	23	QAC		CSV
20013	240904008	Varun Iyer	MALE	2	EE	8.42	6	23	QAC	23	QAC		CSV
20014	240902009	Rajat Kapoor	MALE	2	ECE	6.94	6	23	QAC	23	QAC		CSV
20015	240913010	Aman Kulkarni	MALE	2	IS	8.15	6	23	QAC	23	QAC		CSV
20016	240915011	Siddharth Nair	MALE	2	PE	6.94	6	23	QAC	23	QAC		CSV
20017	240914012	Harsh Kapoor	MALE	2	MT	6.94	6	23	QAC	23	QAC		CSV

- BLOCKS

id	number	gender	floors	roomsPerFloor
1	8	FEMALE	8	30
2	10	MALE	8	30
3	13	FEMALE	8	30
4	14	MALE	8	30
5	15	MALE	8	30
6	18	MALE	8	30
7	19	MALE	8	30
8	20	MALE	8	30
9	21	FEMALE	8	30
10	22	FEMALE	8	30
11	23	MALE	8	30

- ROOM_TYPES

id	code	name	capacity
1	SA	Single Attached Non-AC	1
2	SAC	Single Attached AC	1
3	SC	Single Non-Attached Non-AC	1
4	DA	Double Attached Non-AC	2
5	DAC	Double Attached AC	2
6	DC	Double Non-Attached Non-AC	2
7	QAC	Quadruple Attached AC	4

- ROOMS

id	blockId	roomTypeId	floor	roomNumber	occupiedBeds
1	1	4	1	101	0
2	1	4	1	102	0
3	1	4	1	103	0
4	1	4	1	104	0
5	1	4	1	105	0
6	1	4	1	106	0
7	1	4	1	107	0
8	1	4	1	108	0
9	1	4	1	109	0
10	1	4	1	110	0
11	1	4	1	111	0
12	1	4	1	112	0

5. DDL commands & PL/SQL Procedures / Functions / Triggers

Database	PostgreSQL (Neon serverless)
ORM	Prisma ORM (schema.prisma)
App Stack	Next.js 14 (App Router)
SQL Files	dbms_project_showcase.sql · db_report_queries.sql
Schema File	prisma/schema.prisma

DDL — Data Definition Language & Table Creation

All tables are defined in prisma/schema.prisma and translated to PostgreSQL DDL by Prisma. Raw DDL equivalents are shown below.

allotment_sessions

Central hub table. All other tables reference this via sessionId.

```
CREATE TABLE "allotment_sessions" (
    "id"                    SERIAL          NOT NULL,
    "name"                  TEXT            NOT NULL,
    "status"                "SessionStatus" NOT NULL DEFAULT 'DRAFT',
    "createdAt"             TIMESTAMP(3)    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "algoMode"              TEXT            NOT NULL DEFAULT 'preference_only',
    "portalStatus"          "PortalStatus"  NOT NULL DEFAULT 'CLOSED',
    "blockCutoffsEnabled"   BOOLEAN         NOT NULL DEFAULT false,
    "maxCgpaDiffEnabled"    BOOLEAN         NOT NULL DEFAULT false,
    "maxCgpaDiff"           DOUBLE PRECISION,           -- nullable
    CONSTRAINT "allotment_sessions_pkey" PRIMARY KEY ("id")
);
```

blocks

```
CREATE TABLE "blocks" (
    "id"           SERIAL      NOT NULL,
    "number"       INTEGER     NOT NULL,
    "gender"       "Gender"    NOT NULL,
    "floors"       INTEGER     NOT NULL DEFAULT 8,
    "roomsPerFloor" INTEGER    NOT NULL DEFAULT 30,
    CONSTRAINT "blocks_pkey"   PRIMARY KEY ("id"),
    CONSTRAINT "blocks_number_key" UNIQUE ("number")
);
```

room_types

```
CREATE TABLE "room_types" (
    "id"       SERIAL  NOT NULL,
    "code"     TEXT    NOT NULL,
    "name"     TEXT    NOT NULL,
    "capacity"  INTEGER NOT NULL,
    CONSTRAINT "room_types_pkey"     PRIMARY KEY ("id"),
    CONSTRAINT "room_types_code_key" UNIQUE ("code")
);
```

rooms

```
CREATE TABLE "rooms" (
    "id"            SERIAL  NOT NULL,
    "blockId"       INTEGER NOT NULL,
    "roomTypeId"    INTEGER NOT NULL,
    "floor"         INTEGER NOT NULL,
    "roomNumber"    TEXT    NOT NULL,
    "occupiedBeds"  INTEGER NOT NULL DEFAULT 0,
    CONSTRAINT "rooms_pkey"                 PRIMARY KEY ("id"),
    CONSTRAINT "rooms_blockId_roomNumber_key"  UNIQUE ("blockId", "roomNumber"),
    CONSTRAINT "rooms_blockId_fkey"
        FOREIGN KEY ("blockId")    REFERENCES "blocks"("id"),
    CONSTRAINT "rooms_roomTypeId_fkey"
        FOREIGN KEY ("roomTypeId") REFERENCES "room_types"("id")
);
```

students

```
CREATE TABLE "students" (
    "id"                INTEGER         NOT NULL,
    "regNo"             TEXT            NOT NULL,
    "name"              TEXT            NOT NULL,
    "gender"            "Gender"        NOT NULL,
    "year"              INTEGER         NOT NULL,
    "department"        TEXT            NOT NULL,
    "cgpa"              DOUBLE PRECISION NOT NULL,
    "sessionId"         INTEGER         NOT NULL,
    "pref1BlockNum"     INTEGER,                    -- nullable
    "pref1RoomTypeCode" TEXT,                       -- nullable
    "pref2BlockNum"     INTEGER,                    -- nullable
    "pref2RoomTypeCode" TEXT,                       -- nullable
    "password"          TEXT,                       -- nullable (CSV students have none)
    "source"            "StudentSource" NOT NULL DEFAULT 'CSV',
    CONSTRAINT "students_pkey"                  PRIMARY KEY ("id"),
    CONSTRAINT "students_regNo_sessionId_key" UNIQUE ("regNo", "sessionId"),
    CONSTRAINT "students_sessionId_fkey"
        FOREIGN KEY ("sessionId") REFERENCES "allotment_sessions"("id") ON DELETE CASCADE
);
```

student_groups

```
CREATE TABLE "student_groups" (
    "id"            SERIAL          NOT NULL,
    "avgCgpa"       DOUBLE PRECISION NOT NULL,
    "size"          INTEGER         NOT NULL,
    "pref1BlockId"    INTEGER         NOT NULL,
    "pref1RoomTypeId" INTEGER         NOT NULL,
    "pref2BlockId"    INTEGER         NOT NULL,
    "pref2RoomTypeId" INTEGER         NOT NULL,
    "sessionId"       INTEGER         NOT NULL,
    CONSTRAINT "student_groups_pkey" PRIMARY KEY ("id"),
    CONSTRAINT "student_groups_sessionId_fkey"
        FOREIGN KEY ("sessionId")       REFERENCES "allotment_sessions"("id") ON DELETE CASCADE,
    CONSTRAINT "student_groups_pref1BlockId_fkey"
        FOREIGN KEY ("pref1BlockId")    REFERENCES "blocks"("id"),
    CONSTRAINT "student_groups_pref1RoomTypeId_fkey"
        FOREIGN KEY ("pref1RoomTypeId") REFERENCES "room_types"("id"),
    CONSTRAINT "student_groups_pref2BlockId_fkey"
        FOREIGN KEY ("pref2BlockId")    REFERENCES "blocks"("id"),
    CONSTRAINT "student_groups_pref2RoomTypeId_fkey"
        FOREIGN KEY ("pref2RoomTypeId") REFERENCES "room_types"("id")
);
```

portal_groups

```
CREATE TABLE "portal_groups" (
    "id"            SERIAL      NOT NULL,
    "sessionId"     INTEGER     NOT NULL,
    "inviteCode"    TEXT        NOT NULL,
    "leaderId"      INTEGER     NOT NULL,
    "pref1BlockId"  INTEGER,                -- nullable until submitted
    "pref1RoomTypeId" INTEGER,              -- nullable
    "pref2BlockId"  INTEGER,                -- nullable
    "pref2RoomTypeId" INTEGER,              -- nullable
    "isSubmitted"   BOOLEAN     NOT NULL DEFAULT false,
    "createdAt"     TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT "portal_groups_pkey"          PRIMARY KEY ("id"),
    CONSTRAINT "portal_groups_inviteCode_key" UNIQUE ("inviteCode"),
    CONSTRAINT "portal_groups_sessionId_fkey"
        FOREIGN KEY ("sessionId") REFERENCES "allotment_sessions"("id") ON DELETE CASCADE,
    CONSTRAINT "portal_groups_leaderId_fkey"
        FOREIGN KEY ("leaderId")  REFERENCES "students"("id") ON DELETE CASCADE
);
```

allotments

```
CREATE TABLE "allotments" (
    "id"        SERIAL              NOT NULL,
    "studentId" INTEGER             NOT NULL,
    "roomId"    INTEGER,                        -- nullable (WAITLISTED/OFF_CAMPUS have no room)
    "groupId"   INTEGER             NOT NULL,
    "round"     INTEGER             NOT NULL,
    "status"    "AllotmentStatus"   NOT NULL,
    "sessionId" INTEGER             NOT NULL,
    CONSTRAINT "allotments_pkey"                          PRIMARY KEY ("id"),
    CONSTRAINT "allotments_studentId_sessionId_round_key"  UNIQUE ("studentId", "sessionId", "round"),
    CONSTRAINT "allotments_studentId_fkey"
        FOREIGN KEY ("studentId") REFERENCES "students"("id")        ON DELETE CASCADE,
    CONSTRAINT "allotments_roomId_fkey"
        FOREIGN KEY ("roomId")    REFERENCES "rooms"("id"),
    CONSTRAINT "allotments_groupId_fkey"
        FOREIGN KEY ("groupId")   REFERENCES "student_groups"("id")  ON DELETE CASCADE,
    CONSTRAINT "allotments_sessionId_fkey"
        FOREIGN KEY ("sessionId") REFERENCES "allotment_sessions"("id") ON DELETE CASCADE
);
```

group_members

```
CREATE TABLE "group_members" (
    "id"        SERIAL  NOT NULL,
    "groupId"   INTEGER NOT NULL,
    "studentId" INTEGER NOT NULL,
    CONSTRAINT "group_members_pkey"             PRIMARY KEY ("id"),
    CONSTRAINT "group_members_groupId_studentId_key" UNIQUE ("groupId", "studentId"),
    CONSTRAINT "group_members_groupId_fkey"
        FOREIGN KEY ("groupId")   REFERENCES "student_groups"("id") ON DELETE CASCADE,
    CONSTRAINT "group_members_studentId_fkey"
        FOREIGN KEY ("studentId") REFERENCES "students"("id")       ON DELETE CASCADE
);
```

portal_group_members

```
CREATE TABLE "portal_group_members" (
    "id"           SERIAL  NOT NULL,
    "portalGroupId" INTEGER NOT NULL,
    "studentId"     INTEGER NOT NULL,
    CONSTRAINT "portal_group_members_pkey" PRIMARY KEY ("id"),
    CONSTRAINT "portal_group_members_studentId_portalGroupId_key"
        UNIQUE ("studentId", "portalGroupId"),
    CONSTRAINT "portal_group_members_portalGroupId_fkey"
        FOREIGN KEY ("portalGroupId") REFERENCES "portal_groups"("id") ON DELETE CASCADE,
    CONSTRAINT "portal_group_members_studentId_fkey"
        FOREIGN KEY ("studentId")     REFERENCES "students"("id")      ON DELETE CASCADE
);
```

block_room_configs

```
CREATE TABLE "block_room_configs" (
    "id"           SERIAL  NOT NULL,
    "blockId"       INTEGER NOT NULL,
    "roomTypeId"    INTEGER NOT NULL,
    "roomsPerFloor" INTEGER NOT NULL,
    CONSTRAINT "block_room_configs_pkey"               PRIMARY KEY ("id"),
    CONSTRAINT "block_room_configs_blockId_roomTypeId_key" UNIQUE ("blockId", "roomTypeId"),
    CONSTRAINT "block_room_configs_blockId_fkey"
        FOREIGN KEY ("blockId")    REFERENCES "blocks"("id"),
    CONSTRAINT "block_room_configs_roomTypeId_fkey"
        FOREIGN KEY ("roomTypeId") REFERENCES "room_types"("id")
);
```

block_cutoffs

```
CREATE TABLE "block_cutoffs" (
    "id"        SERIAL          NOT NULL,
    "sessionId" INTEGER         NOT NULL,
    "blockId"   INTEGER         NOT NULL,
    "minCgpa"   DOUBLE PRECISION NOT NULL,
    CONSTRAINT "block_cutoffs_pkey"              PRIMARY KEY ("id"),
    CONSTRAINT "block_cutoffs_sessionId_blockId_key" UNIQUE ("sessionId", "blockId"),
    CONSTRAINT "block_cutoffs_sessionId_fkey"
        FOREIGN KEY ("sessionId") REFERENCES "allotment_sessions"("id") ON DELETE CASCADE,
    CONSTRAINT "block_cutoffs_blockId_fkey"
        FOREIGN KEY ("blockId")   REFERENCES "blocks"("id")            ON DELETE CASCADE
);
```

allotment_audits

```
CREATE TABLE "allotment_audits" (
    "id"        SERIAL      NOT NULL,
    "studentId" INTEGER     NOT NULL,
    "oldStatus" TEXT        NOT NULL,
    "newStatus" TEXT        NOT NULL,
    "changedAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT "allotment_audits_pkey" PRIMARY KEY ("id")
    -- Note: no FK to students — intentional, audits persist independently
);
```

session_audit_log (trigger support table)

```
CREATE TABLE IF NOT EXISTS session_audit_log (
    id          SERIAL PRIMARY KEY,
    "sessionId" INTEGER     NOT NULL,
    old_status  VARCHAR(50),
    new_status  VARCHAR(50),
    changed_at  TIMESTAMP   DEFAULT CURRENT_TIMESTAMP
);
```

Basic DML — INSERT, SELECT, UPDATE, DELETE

INSERT

— New Allotment Session

```
INSERT INTO allotment_sessions (name, status, "createdAt")
VALUES ('2024-25 Odd Semester', 'DRAFT', CURRENT_TIMESTAMP)
RETURNING id, name, status;
```

— Batch CSV Student Upload (ON CONFLICT DO NOTHING).

```
INSERT INTO students ("regNo", name, gender, year, department, cgpa, source, "sessionId")
VALUES
    ('21CSE001', 'Alice Sharma',   'FEMALE', 3, 'CSE',  9.2, 'CSV', 1),
    ('21ECE042', 'Bob Patel',      'MALE',   2, 'ECE',  8.7, 'CSV', 1),
    ('21MECH07', 'Carol Reddy',    'FEMALE', 4, 'MECH', 7.8, 'CSV', 1)
ON CONFLICT ("regNo", "sessionId") DO NOTHING
RETURNING "regNo", name, cgpa;
```

— Portal Student Self-Registration

```
INSERT INTO students ("regNo", name, gender, year, department, cgpa, password, source, "sessionId")
VALUES (
    '21CSE099', 'Demo Student', 'MALE', 2, 'CSE', 8.5,
    '$2b$10$hashedpassword...', 'PORTAL', 1
)
ON CONFLICT ("regNo", "sessionId") DO NOTHING
RETURNING id, "regNo", name, source;
```

SELECT

— Fetch a Student by RegNo (Login Lookup)

```
SELECT id, "regNo", name, gender, cgpa, source, password
FROM students
WHERE "regNo" = '21CSE001'
  AND "sessionId" = 1
LIMIT 1;
```

— Top 10 Students by CGPA in Latest Session

```
SELECT "regNo", name, gender, year, department, cgpa, source
FROM students
WHERE "sessionId" = (SELECT MAX(id) FROM allotment_sessions)
ORDER BY cgpa DESC
LIMIT 10;
```

— All Blocks with Room Type Breakdown

```
SELECT b.number AS block, b.gender, rt.name AS room_type, brc."roomsPerFloor"
FROM blocks b
JOIN block_room_configs brc ON b.id = brc."blockId"
JOIN room_types rt          ON brc."roomTypeId" = rt.id
ORDER BY b.number, rt.name;
```

— Portal Students Only

```
SELECT id, "regNo", name, gender, source
FROM students
WHERE source = 'PORTAL'
ORDER BY id DESC
LIMIT 5;
```

UPDATE

— Open Student Portal

```
UPDATE allotment_sessions
SET "portalStatus" = 'OPEN'
WHERE id = 1
RETURNING id, name, "portalStatus";
```

— Lock Group Preferences

```
UPDATE portal_groups
SET
    "pref1BlockId"    = 3,
    "pref1RoomTypeId" = 1,
    "pref2BlockId"    = 5,
    "pref2RoomTypeId" = 2,
    "isSubmitted"     = true
WHERE id = 42
RETURNING id, "isSubmitted", "pref1BlockId", "pref2BlockId";
```

— Set Algorithm Rules (Block Cutoffs Flag)

```
UPDATE allotment_sessions
SET "blockCutoffsEnabled" = true,
    "maxCgpaDiffEnabled"  = true,
    "maxCgpaDiff"         = 1.5
WHERE id = 1;
```

DELETE

— Student Leaves a Portal Group

```
DELETE FROM portal_group_members
WHERE "studentId"     = 101
  AND "portalGroupId" = 7
RETURNING *;
```

— Disallow Allotment (Admin Override)

```
-- Step 1: Mark allotment as DISALLOWED
UPDATE allotments
SET status = 'DISALLOWED'
WHERE id = 55
RETURNING id, "studentId", status;


-- Step 2: Write audit record
INSERT INTO allotment_audits ("studentId", "oldStatus", "newStatus", "changedAt")
VALUES (101, 'ALLOTTED', 'DISALLOWED', CURRENT_TIMESTAMP);
```

Complex Queries — Multi-Join & Aggregations

Student Dashboard — LEFT JOIN Across 4 Tables

Used by the student dashboard API to show allotment status, room, and block in a single query.

```
SELECT
    s."regNo",  s.name,   s.gender, s.year,
    s.department, s.cgpa,
    a.status        AS allotment_status,
    a.round         AS allotment_round,
    r."roomNumber",
    rt.name         AS room_type,
    b.number        AS block_number,
    b.gender        AS block_gender
FROM students s
LEFT JOIN allotments  a  ON s.id = a."studentId"
LEFT JOIN rooms       r  ON a."roomId" = r.id
LEFT JOIN room_types  rt ON r."roomTypeId" = rt.id
LEFT JOIN blocks      b  ON r."blockId" = b.id
WHERE s."sessionId" = (SELECT MAX(id) FROM allotment_sessions)
ORDER BY s.cgpa DESC;
```

Rooms per Block — Total Count and Capacity

```
SELECT
    b.number  AS block,
    b.gender,
    COUNT(r.id)       AS total_rooms,
    SUM(rt.capacity)  AS total_beds
FROM blocks b
JOIN rooms      r  ON r."blockId"    = b.id
JOIN room_types rt ON r."roomTypeId" = rt.id
GROUP BY b.number, b.gender
ORDER BY b.number;
```

Department-wise CGPA Aggregation

```
SELECT
    department,
    COUNT(*)                          AS student_count,
    ROUND(AVG(cgpa)::numeric, 2)        AS avg_cgpa,
    MAX(cgpa)                         AS top_cgpa,
    MIN(cgpa)                         AS lowest_cgpa
FROM students
WHERE "sessionId" = (SELECT MAX(id) FROM allotment_sessions)
GROUP BY department
ORDER BY avg_cgpa DESC;
```

Block Occupancy Report — Allotted Students + Avg CGPA

```
SELECT
    b.number  AS block_number,
    b.gender,
    COUNT(a.id)                       AS allotted_students,
    ROUND(AVG(s.cgpa)::numeric, 2)      AS avg_cgpa
FROM blocks b
JOIN rooms      r  ON r."blockId"    = b.id
JOIN allotments a  ON a."roomId"     = r.id
JOIN students   s  ON a."studentId"  = s.id
WHERE a.status = 'ALLOTTED'
GROUP BY b.number, b.gender
ORDER BY b.number;
```

All Sessions — Student, Group & Allotment Counts (Correlated Subqueries)

```
SELECT
    s.id,
    s.name,
    s.status,
    s."portalStatus",
    (SELECT COUNT(*) FROM students       st WHERE st."sessionId" = s.id) AS student_count,
    (SELECT COUNT(*) FROM student_groups sg WHERE sg."sessionId" = s.id) AS group_count,
    (SELECT COUNT(*) FROM allotments      a WHERE  a."sessionId" = s.id) AS allotment_count
FROM allotment_sessions s
ORDER BY s."createdAt" DESC;
```

Algorithm Group Loader — Groups Sorted by Avg CGPA + Seniority

Used by the allotment engine to determine priority ordering. Groups with higher avgCgpa rank first; ties broken by smallest year (senior students).

```
SELECT
    sg.id,
    sg."avgCgpa",
    sg.size,
    sg."pref1BlockId",
    sg."pref1RoomTypeId",
    sg."pref2BlockId",
    sg."pref2RoomTypeId",
    MIN(s.year) AS min_year         -- used as tiebreaker (lower = more senior)
FROM student_groups sg
JOIN group_members gm ON sg.id = gm."groupId"
JOIN students      s  ON gm."studentId" = s.id
WHERE sg."sessionId" = (SELECT MAX(id) FROM allotment_sessions)
GROUP BY sg.id
ORDER BY sg."avgCgpa" DESC, min_year ASC;
```

Portal → Student Groups Sync (Engine Pre-Step)

Computes aggregated student group data from submitted portal groups, ready for INSERT into student_groups.

```
SELECT
    COUNT(pgm."studentId")                AS size,
    ROUND(AVG(s.cgpa)::numeric, 2)          AS "avgCgpa",
    pg."pref1BlockId",
    pg."pref1RoomTypeId",
    pg."pref2BlockId",
    pg."pref2RoomTypeId",
    pg."sessionId"
FROM portal_groups      pg
JOIN portal_group_members pgm ON pg.id          = pgm."portalGroupId"
JOIN students             s   ON pgm."studentId" = s.id
WHERE pg."sessionId"   = (SELECT MAX(id) FROM allotment_sessions)
  AND pg."isSubmitted" = true
GROUP BY pg.id;
```

Full Allotment Results Export (Admin)

Used by the export API to generate downloadable CSV results for the admin.

```
SELECT
    s."regNo",
    s.name,
    s.gender,
    s.year,
    s.department,
    s.cgpa,
    a.status        AS allotment_status,
    a.round,
    r."roomNumber",
    rt.name         AS room_type,
    rt.capacity,
    b.number        AS block_number,
    b.gender        AS block_gender
FROM allotments a
JOIN students   s  ON a."studentId"  = s.id
LEFT JOIN rooms       r  ON a."roomId"     = r.id
LEFT JOIN room_types  rt ON r."roomTypeId" = rt.id
LEFT JOIN blocks      b  ON r."blockId"    = b.id
WHERE a."sessionId" = 1
ORDER BY b.number, r."roomNumber", s.cgpa DESC;
```

Views

Location: scripts/setup-db-extensions.js and dbms_project_showcase.sql Part 1A

vw_room_vacancies — Live Room Vacancy Monitor

Joins 4 tables to compute occupied vs. vacant beds per room in real time. Used by the admin UI and allotment engine to find available rooms.

```
CREATE OR REPLACE VIEW vw_room_vacancies AS
SELECT
    b.number                                AS block_number,
    b.gender                                AS block_gender,
    r."roomNumber",
    r.floor,
    rt.name                                 AS room_type,
    rt.capacity                             AS total_capacity,
    COALESCE(COUNT(a.id), 0)                AS occupied_beds,
    (rt.capacity - COALESCE(COUNT(a.id), 0))        AS vacant_beds
FROM rooms r
JOIN blocks      b  ON r."blockId"    = b.id
JOIN room_types  rt ON r."roomTypeId" = rt.id
LEFT JOIN allotments a
    ON a."roomId" = r.id AND a.status = 'ALLOTTED'
GROUP BY
    b.number, b.gender, r."roomNumber", r.floor,
    rt.name, rt.capacity;


-- Query examples:
-- SELECT * FROM vw_room_vacancies WHERE vacant_beds > 0 ORDER BY block_number;
-- SELECT block_number, SUM(vacant_beds) AS total_vacant FROM vw_room_vacancies GROUP BY block_number;
```

Triggers

Location: scripts/setup-db-extensions.js (Lines ~35–65) and dbms_project_showcase.sql Part 1B

trg_session_status — Session Status Audit Log

Fires AFTER UPDATE on allotment_sessions. Any time the status column changes (e.g. DRAFT → ROUND1_LOCKED), the old and new values are written immutably to session_audit_log — independent of the Node.js application layer.

```
-- Step 1: Audit support table (created once)
CREATE TABLE IF NOT EXISTS session_audit_log (
    id          SERIAL PRIMARY KEY,
    "sessionId" INTEGER      NOT NULL,
    old_status  VARCHAR(50),
    new_status  VARCHAR(50),
    changed_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);

-- Step 2: Trigger function
CREATE OR REPLACE FUNCTION log_session_status_change()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status <> OLD.status THEN
        INSERT INTO session_audit_log ("sessionId", old_status, new_status)
        VALUES (OLD.id, OLD.status, NEW.status);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Step 3: Bind trigger to table
DROP TRIGGER IF EXISTS trg_session_status ON allotment_sessions;
CREATE TRIGGER trg_session_status
AFTER UPDATE ON allotment_sessions
FOR EACH ROW
EXECUTE FUNCTION log_session_status_change();

-- Verify: read the audit log
-- SELECT * FROM session_audit_log ORDER BY changed_at DESC;
```

Stored Procedures

Location: scripts/setup-db-extensions.js (Lines ~68–80) and dbms_project_showcase.sql Part 1C

proc_reset_session_allotments — Hard Reset a Session

Atomically deletes all allotment rows for a session and resets its status back to DRAFT. Used by the admin 'Reset Session' button. Combines DELETE + UPDATE + diagnostic logging in one database-side call.

```
CREATE OR REPLACE PROCEDURE proc_reset_session_allotments(target_session_id INT)
LANGUAGE plpgsql AS $$
DECLARE
    deleted_count INT;
BEGIN
    -- Step 1: Remove all allotment records for this session
    DELETE FROM allotments WHERE "sessionId" = target_session_id;
    GET DIAGNOSTICS deleted_count = ROW_COUNT;

    -- Step 2: Reset the session lifecycle status
    UPDATE allotment_sessions
    SET status = 'DRAFT'
    WHERE id = target_session_id;
```

```
    -- Step 3: Emit a server-side notice (visible in psql / Neon console)
    RAISE NOTICE 'Reset complete. % allotment rows deleted for session %.',
        deleted_count, target_session_id;
END;
$$;

-- Invocation:
CALL proc_reset_session_allotments(1);

-- Or with dynamic latest session:
CALL proc_reset_session_allotments((SELECT MAX(id) FROM allotment_sessions));
```

Cursors

Location: dbms_project_showcase.sql Part 1D

generate_waitlist_report — Ranked Waitlist Report via Cursor

Uses an explicit CURSOR to iterate through all WAITLISTED students row-by-row in CGPA order and builds a formatted ranked text report. Demonstrates memory-efficient row-by-row processing using OPEN / FETCH / CLOSE semantics.

```
CREATE OR REPLACE FUNCTION generate_waitlist_report(target_session_id INT)
RETURNS TEXT
LANGUAGE plpgsql AS $$
DECLARE
    -- Declare a named cursor with its driving query
    waitlist_cursor CURSOR FOR
        SELECT s.name, s."regNo", s.cgpa
        FROM allotments a
        JOIN students s ON a."studentId" = s.id
        WHERE a."sessionId" = target_session_id
          AND a.status      = 'WAITLISTED'
        ORDER BY s.cgpa DESC;

    rec         RECORD;
    report      TEXT := e'--- WAITLIST REPORT ---\n';
    row_rank    INT  := 1;
BEGIN
    OPEN waitlist_cursor;               -- Open the cursor

    LOOP
        FETCH waitlist_cursor INTO rec; -- Fetch next row
        EXIT WHEN NOT FOUND;            -- Exit when exhausted

        report := report
            || row_rank        || '. '
            || rec.name        || ' ('
            || rec."regNo"     || ') — CGPA: '
            || rec.cgpa        || e'\n';

        row_rank := row_rank + 1;
    END LOOP;

    CLOSE waitlist_cursor;              -- Release the cursor

    RETURN report;
END;
$$;

-- Invocation:
SELECT generate_waitlist_report(1);
SELECT generate_waitlist_report((SELECT MAX(id) FROM allotment_sessions));
```

Atomic Transactions (BEGIN / COMMIT / ROLLBACK)

All critical multi-step operations are wrapped in transactions to guarantee atomicity.

Open Portal — Update + Delete (Atomic)

When admin opens the portal, existing portal group submissions are wiped atomically so students start fresh. If either step fails, both are rolled back.

```
BEGIN;
   -- Step 1: Set portal to OPEN
   UPDATE allotment_sessions
   SET "portalStatus" = 'OPEN'
   WHERE id = (SELECT MAX(id) FROM allotment_sessions);

   -- Step 2: Erase stale portal group submissions atomically
   DELETE FROM portal_groups
   WHERE "sessionId" = (SELECT MAX(id) FROM allotment_sessions);
COMMIT;
```

Upsert Block CGPA Cutoffs (ON CONFLICT DO UPDATE)

Inserts a new cutoff or updates it if one already exists for the same session+block pair. Safe to call multiple times.

```
BEGIN;
   -- Enable cutoff flags on session
   UPDATE allotment_sessions
   SET "blockCutoffsEnabled" = true,
      "maxCgpaDiffEnabled"  = true,
      "maxCgpaDiff"         = 1.5
   WHERE id = 1;

   -- Upsert the cutoff value
   INSERT INTO block_cutoffs ("sessionId", "blockId", "minCgpa")
   VALUES (1, 3, 8.0)
   ON CONFLICT ("sessionId", "blockId")
   DO UPDATE SET "minCgpa" = EXCLUDED."minCgpa"
   RETURNING "sessionId", "blockId", "minCgpa";
COMMIT;
```

Create Portal Group + Add Member (Full Flow)

```
BEGIN;
   -- Step 1: Create the portal group with invite code
   INSERT INTO portal_groups ("inviteCode", "leaderId", "sessionId", "isSubmitted")
   VALUES (
      'GRPCODE42',
      101,        -- leaderId
      1,          -- sessionId
      false
   )
   RETURNING id, "inviteCode", "leaderId";

   -- Step 2: Add a second member to the group
   INSERT INTO portal_group_members ("portalGroupId", "studentId")
   VALUES (
      (SELECT id FROM portal_groups WHERE "inviteCode" = 'GRPCODE42'),
      102         -- second student
   );

   -- Step 3: Verify
   SELECT pg.id, pg."inviteCode", COUNT(pgm.id) AS member_count
   FROM portal_groups pg
   LEFT JOIN portal_group_members pgm ON pg.id = pgm."portalGroupId"
   WHERE pg."inviteCode" = 'GRPCODE42'
```

```
    GROUP BY pg.id;
COMMIT;
```

Batch Allotment Write-back (Engine Final Step)

After the JS allotment engine computes all assignments in memory, the final results are written atomically. All allotments, room occupancies, and session status update together or not at all.

```
BEGIN;
    -- Step 1: Bulk insert computed allotments
    INSERT INTO allotments ("studentId", "roomId", "groupId", "round", status, "sessionId")
    VALUES
        (101, 501, 10, 1, 'ALLOTTED',   1),
        (102, 501, 10, 1, 'ALLOTTED',   1),
        (103, NULL,  11, 1, 'WAITLISTED', 1),
        (104, NULL,  12, 1, 'OFF_CAMPUS', 1);

    -- Step 2: Update room occupancy counters
    UPDATE rooms SET "occupiedBeds" = "occupiedBeds" + 2 WHERE id = 501;

    -- Step 3: Mark session as ROUND1_DONE
    UPDATE allotment_sessions
    SET status = 'ROUND1_DONE'
    WHERE id = 1;
COMMIT;
```

Additional Advanced Queries

Audit Log Viewer (Admin History Page)

```
-- View all session status transitions ordered newest first
SELECT
    sal.id,
    ase.name        AS session_name,
    sal.old_status,
    sal.new_status,
    sal.changed_at
FROM session_audit_log sal
JOIN allotment_sessions ase ON sal."sessionId" = ase.id
ORDER BY sal.changed_at DESC
LIMIT 50;
```

Block Vacancy Summary (from View)

```
-- Total vacant beds per block
SELECT
    block_number,
    block_gender,
    SUM(total_capacity)  AS total_beds,
    SUM(occupied_beds)   AS occupied,
    SUM(vacant_beds)     AS vacant
FROM vw_room_vacancies
GROUP BY block_number, block_gender
ORDER BY block_number;

-- Show only rooms with available space
SELECT * FROM vw_room_vacancies
WHERE vacant_beds > 0
ORDER BY block_number, "roomNumber";
```

Allotment Status Distribution per Session

```
SELECT
    "sessionId",
    status,
    COUNT(*) AS count
FROM allotments
GROUP BY "sessionId", status
ORDER BY "sessionId", status;
```

Students Without Any Allotment (Unprocessed)

```
SELECT s.id, s."regNo", s.name, s.cgpa, s.department
FROM students s
LEFT JOIN allotments a
    ON s.id = a."studentId" AND s."sessionId" = a."sessionId"
WHERE a.id IS NULL
  AND s."sessionId" = (SELECT MAX(id) FROM allotment_sessions)
ORDER BY s.cgpa DESC;
```

Portal Groups with Member Counts and Submission Status

```
SELECT
    pg.id,
    pg."inviteCode",
    s.name          AS leader_name,
    COUNT(pgm.id)   AS member_count,
    pg."isSubmitted",
    b1.number       AS pref1_block,
    rt1.code        AS pref1_room_type,
    b2.number       AS pref2_block,
    rt2.code        AS pref2_room_type
FROM portal_groups pg
JOIN students             s    ON pg."leaderId"        = s.id
LEFT JOIN portal_group_members pgm ON pg.id            = pgm."portalGroupId"
LEFT JOIN blocks          b1   ON pg."pref1BlockId"    = b1.id
LEFT JOIN room_types      rt1  ON pg."pref1RoomTypeId" = rt1.id
LEFT JOIN blocks          b2   ON pg."pref2BlockId"    = b2.id
LEFT JOIN room_types      rt2  ON pg."pref2RoomTypeId" = rt2.id
WHERE pg."sessionId" = (SELECT MAX(id) FROM allotment_sessions)
GROUP BY pg.id, s.name, b1.number, rt1.code, b2.number, rt2.code
ORDER BY pg."isSubmitted" DESC, member_count DESC;
```

Block Cutoffs Report

```
SELECT
    ase.name    AS session_name,
    b.number    AS block_number,
    b.gender,
    bc."minCgpa"
FROM block_cutoffs bc
JOIN allotment_sessions ase ON bc."sessionId" = ase.id
JOIN blocks             b   ON bc."blockId"   = b.id
ORDER BY ase.id, b.number;
```

6. RESULTS & SNAPSHOTS

6.1. Query Results

- Retrieving block-wise room configuration details with gender, room type, and rooms per floor

```sql
SELECT b.number AS block, b.gender, rt.name AS room_type, brc."roomsPerFloor"
FROM blocks b
JOIN block_room_configs brc ON b.id = brc."blockId"
JOIN room_types rt          ON brc."roomTypeId" = rt.id
ORDER BY b.number, rt.name;
```

Connected (1 query)

Run | Explain | Analyze | 435ms | 25 rows

#	block	gender	room_type	roomsPerFloor
1	8	FEMALE	Double Attached AC	15
2	8	FEMALE	Double Attached Non-AC	15
3	10	MALE	Double Non-Attached Non-AC	10
4	10	MALE	Single Attached AC	10
5	10	MALE	Single Non-Attached Non-AC	10
6	13	FEMALE	Double Attached AC	8
7	13	FEMALE	Double Attached Non-AC	8

- Inserting or Updating Minimum CGPA Cutoff for a Block in the Latest Allotment Session (Upsert Operation)

```sql
BEGIN;
    INSERT INTO block_cutoffs ("sessionId", "blockId", "minCgpa")
    VALUES (
        (SELECT MAX(id) FROM allotment_sessions),
        (SELECT id FROM blocks LIMIT 1),
        8.0
    )
    ON CONFLICT ("sessionId", "blockId")
    DO UPDATE SET "minCgpa" = EXCLUDED."minCgpa"
```

Connected (3 queries)

Run | Explain | Analyze

1: BEGIN | 2: INSERT 1 | 3: ROLLBACK

#	sessionId	blockId	minCgpa
1	6	1	8

- Creating a Session Audit Log Table to Track Status Changes with Timestamps

```sql
-- ---- 1B. TRIGGER: Session Status Audit Log ----
-- Automatically logs every session status change (DRAFT → OPEN → LOCKED).

CREATE TABLE IF NOT EXISTS session_audit_log (
    id          SERIAL PRIMARY KEY,
    "sessionId" INTEGER      NOT NULL,
    old_status  VARCHAR(50),
    new_status  VARCHAR(50),
    changed_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
```

Connected (4 queries)

Run | Explain | Analyze | 354ms | No result

1: CREATE | 2: CREATE | 3: DROP | 4: CREATE

Statement executed successfully

- Resetting a Session by Deleting All Allotments and Reverting Status to Draft Using a Stored Procedure

```sql
-- ---- 1C. STORED PROCEDURE: Hard Reset a Session ----
-- Atomically deletes all allotments and resets session status to DRAFT.

CREATE OR REPLACE PROCEDURE proc_reset_session_allotments(target_session_id INT)
LANGUAGE plpgsql AS $$
DECLARE
    deleted_count INT;
BEGIN
    DELETE FROM allotments WHERE "sessionId" = target_session_id;
```

Connected (1 query)

Run | Explain | Analyze

Statement executed successfully

- Generating a Ranked Waitlist Report of Students Based on CGPA Using a Cursor Function

```sql
-- ---- 1D. CURSOR FUNCTION: Waitlist Report (CGPA order) ----
-- Iterates through waitlisted students row-by-row to build a ranked text report.

CREATE OR REPLACE FUNCTION generate_waitlist_report(target_session_id INT)
RETURNS TEXT LANGUAGE plpgsql AS $$
DECLARE
    waitlist_cursor CURSOR FOR
        SELECT s.name, s."regNo", s.cgpa
        FROM allotments a
```

Connected (2 queries)

Run | Explain | Analyze | 249ms | 1 row

1: CREATE | 2: SELECT 1

#	generate_waitlist_report
1	--- WAITLIST REPORT ---

- Top 10 Students by CGPA from the Most Recent Allotment Session

```sql
-- ---- 2D. Student sample: First 10 students in latest session ----
SELECT "regNo", name, gender, year, department, cgpa, source
FROM students
WHERE "sessionId" = (SELECT MAX(id) FROM allotment_sessions)
ORDER BY cgpa DESC
LIMIT 10;
```

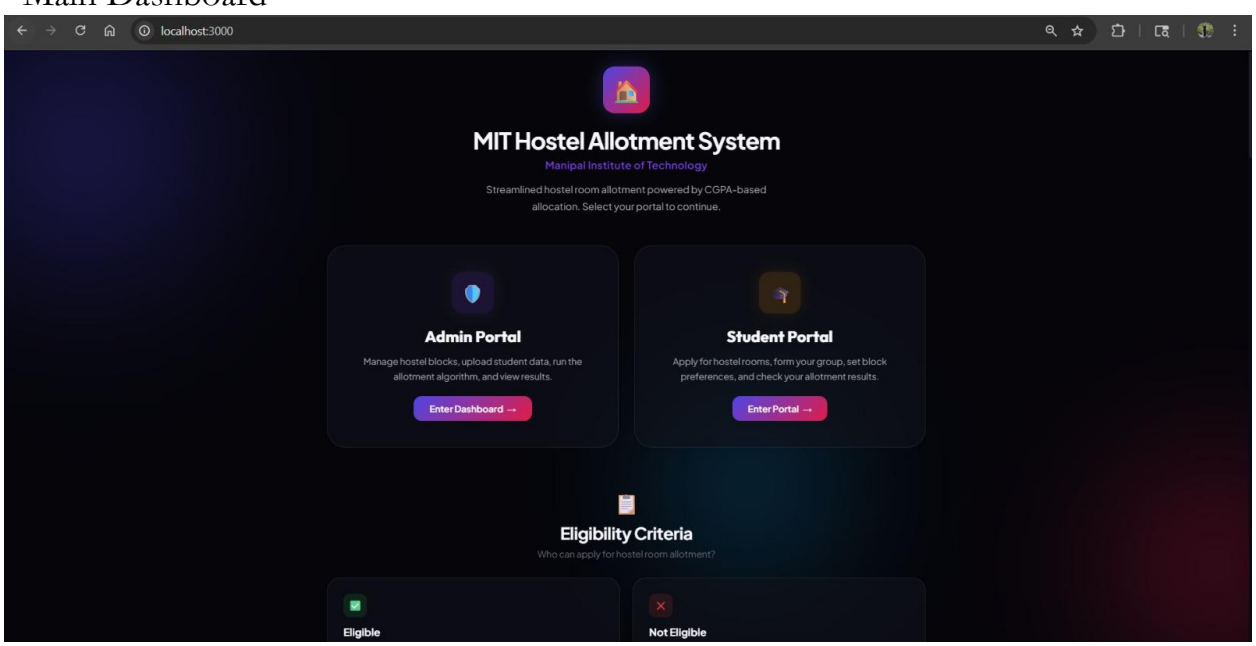
Connected (1 query)

Run | Explain | Analyze | 298ms | 10 rov

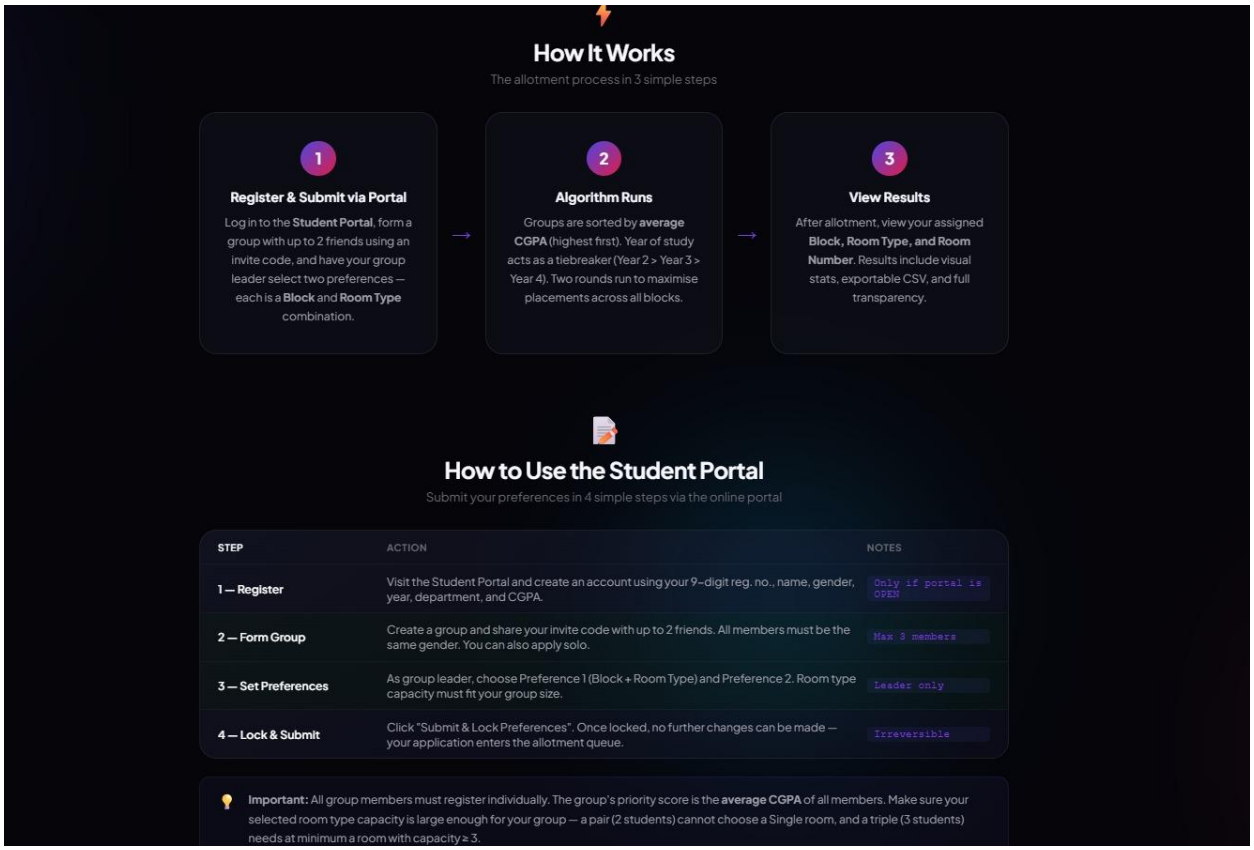
#	regNo	name	gender	year	department	cgpa	source
1	240905910	Aditya Bansal	MALE	2	IT	10	CSV
2	240913848	Raghav Joshi	MALE	2	IS	10	CSV
3	240904881	Pranav Singh	MALE	2	EE	10	CSV
4	240906756	Rajat Bhatt	MALE	2	CCE	10	CSV
5	240903615	Karan Mehta	MALE	2	ME	10	CSV
6	240902584	Nikhil Menon	MALE	2	ECE	10	CSV
7	240902763	Kunal Iyer	MALE	2	ECE	10	CSV

6.2. Website Snapshots

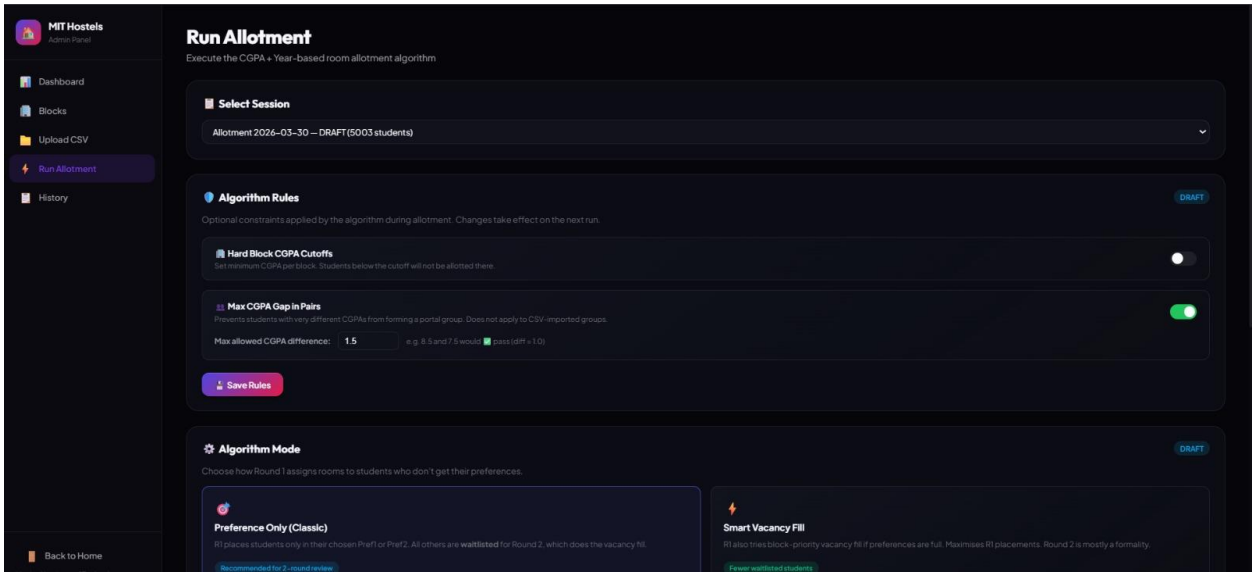
- Main Dashboard



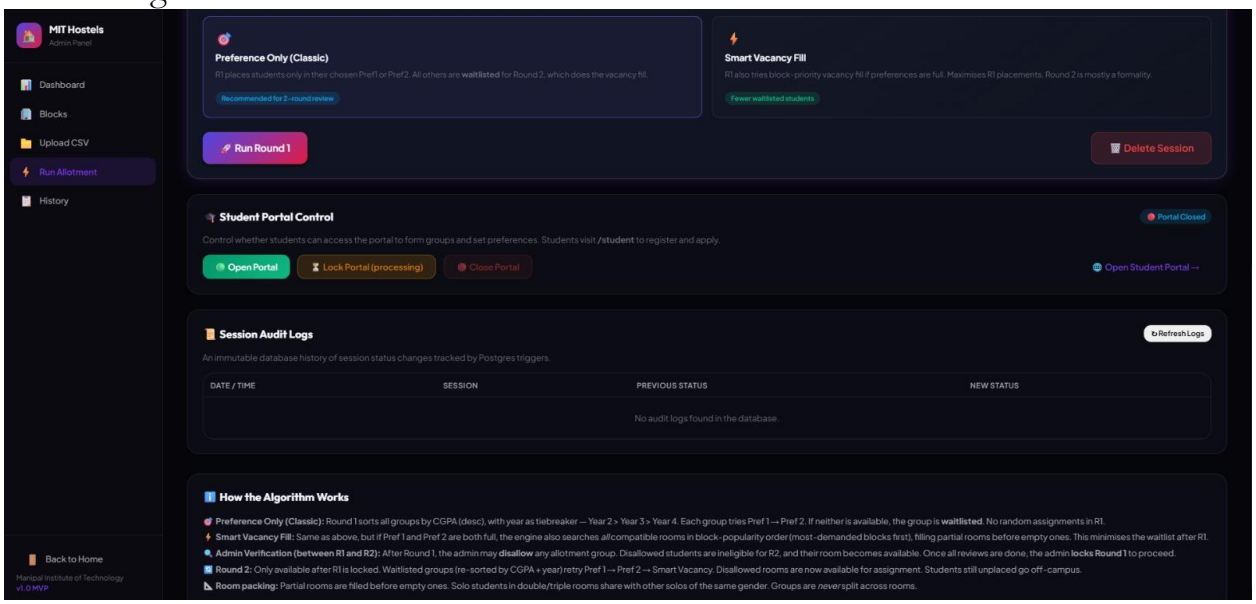
- General Information



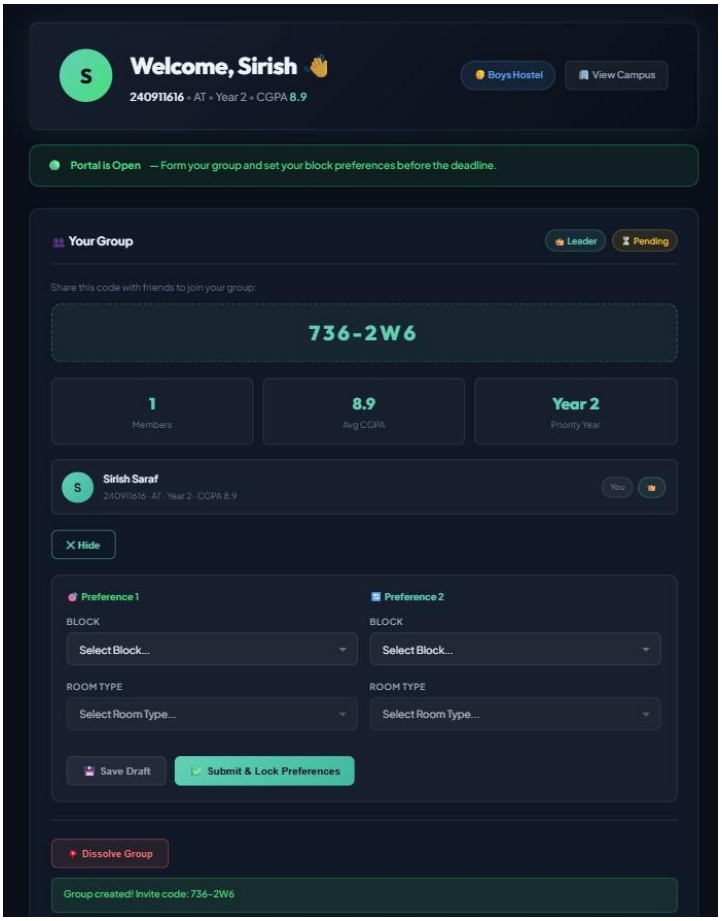
- Select Sessions

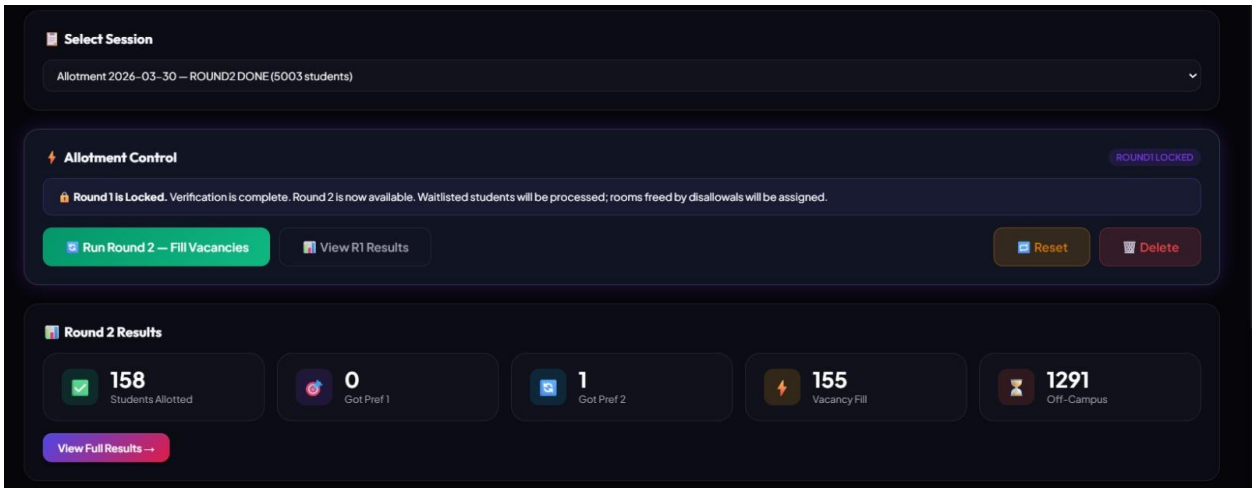


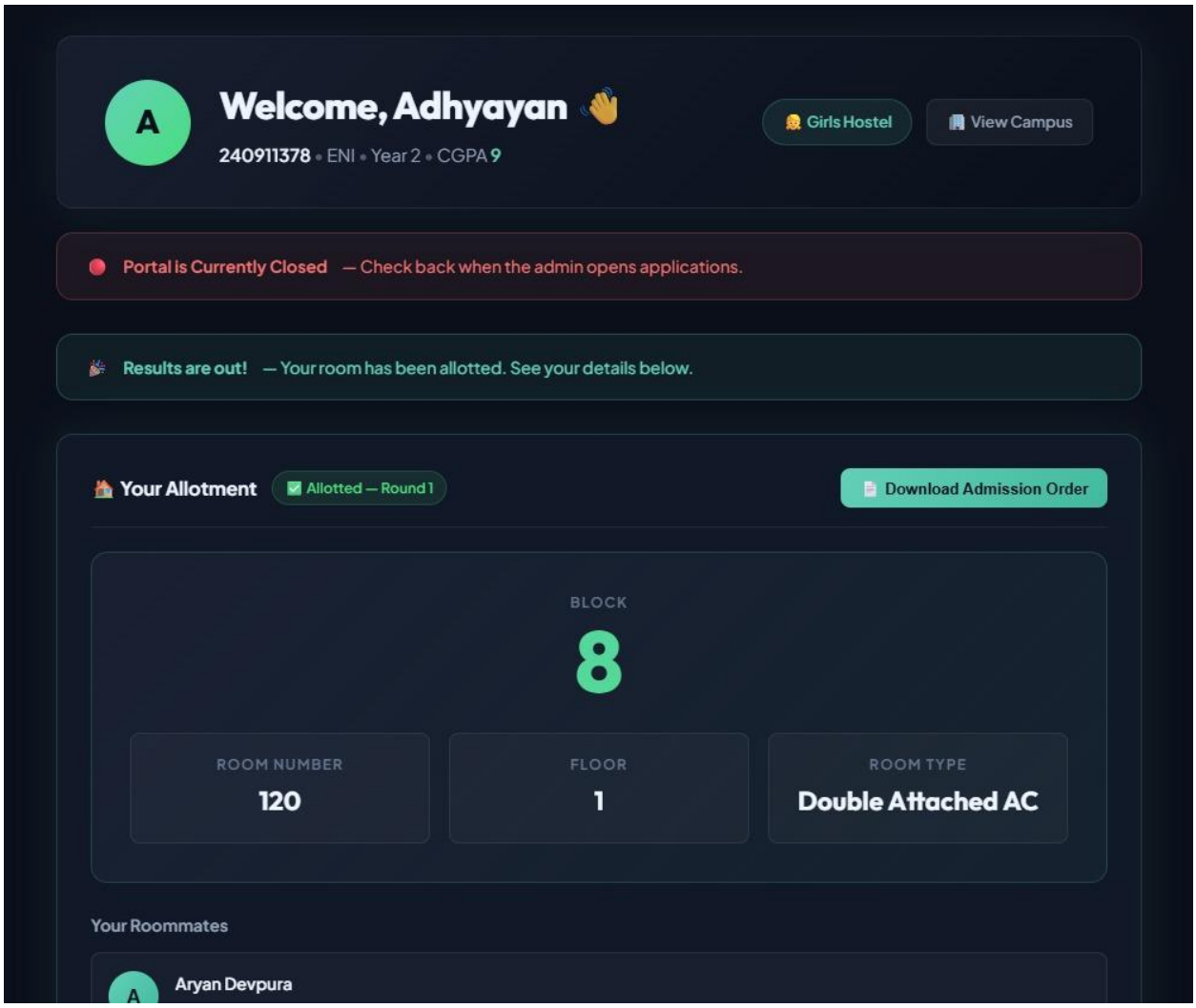
- Select Algorithm and Run Round-1



-




Select Session
Allotment 2026-03-30 — ROUND2 DONE (5003 students)
Allotment Control
ROUND1 LOCKED
Round 1 is Locked. Verification is complete. Round 2 is now available. Waitlisted students will be processed; rooms freed by disallowals will be assigned.
Run Round 2 — Fill Vacancies
View R1 Results
Reset
Delete
Round 2 Results
158
Students Allotted
0
Got Pref 1
1
Got Pref 2
155
Vacancy Fill
1291
Off-Campus
View Full Results →


A
Welcome, Adhyayan
240911378 • ENI • Year 2 • CGPA 9
Girls Hostel
View Campus
Portal Is Currently Closed — Check back when the admin opens applications.
Results are out! — Your room has been allotted. See your details below.
Your Allotment
Allotted — Round 1
Download Admission Order
BLOCK
8
ROOM NUMBER
120
FLOOR
1
ROOM TYPE
Double Attached AC
Your Roommates
Aryan Devpura

7. CONCLUSION, LIMITATIONS & FUTURE WORK

7.1. Conclusion

The Hostel Management System was successfully designed and implemented using a relational database approach, where the underlying database is SQL-based and accessed through Prisma ORM. The project demonstrates effective management of complex relationships between entities such as students, groups, rooms, and allocations using structured schema design. Prisma ORM provides an abstraction layer over SQL, enabling efficient data handling while internally generating the required SQL queries for operations such as joins, filtering, and aggregation. A normalized schema was developed to reduce redundancy and ensure data integrity across all entities. The allocation system effectively assigns rooms based on student preferences and availability, ensuring fairness and efficiency. Overall, the project highlights the importance of database design, normalization, and ORM-based data management in building scalable and maintainable systems.

7.2. Limitations

- The database tries to mimic actual MIT data. However, this still contains a bunch of assumptions due to lack of public information and low access to female blocks despite several requests.
- Monetary transactions are not implemented due to knowledge constraints and complexity in security factors.
- The database design does not extensively utilize performance optimization techniques such as indexing and query tuning.

7.3. Future Work

- Introduce concurrency control mechanisms such as transactions, locking, or isolation levels to handle simultaneous allocation requests. This will prevent issues like race conditions, double allocation of rooms, and inconsistent updates in high-usage scenarios.
- Apply indexing strategies on frequently queried attributes such as student_id, room_id, and session_id to improve query performance, especially for large datasets and complex joins.
- Scaling the solution to enterprise level and pitching it to administration for real world use and monetary gains.
- Enhance the schema to better handle scalability and large datasets, ensuring efficient storage and retrieval even when the number of students and rooms increases significantly.

8. REFERENCES

1. HMS portal: https://hms.manipal.edu
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.
3. PostgreSQL Global Development Group. (2024). PostgreSQL 16.0 Documentation. Retrieved from https://www.postgresql.org/docs/
4. Prisma ORM. (2024). Prisma Documentation - Relational Database Mapping. Retrieved from https://www.prisma.io/docs/
5. Neon Inc. (2024). Neon Serverless Postgres Architecture Documentation. Retrieved from https://neon.tech/docs/
6. Vercel. (2024). Next.js Documentation - App Router and API Routes. Retrieved from https://nextjs.org/docs
7. T. Cormen, C. Leiserson, R. Rivest, and C. Stein. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
8. https://github.com/Siri-shh/PM-Internship-Scheme - SIH Final Project Documentation and Methodology.